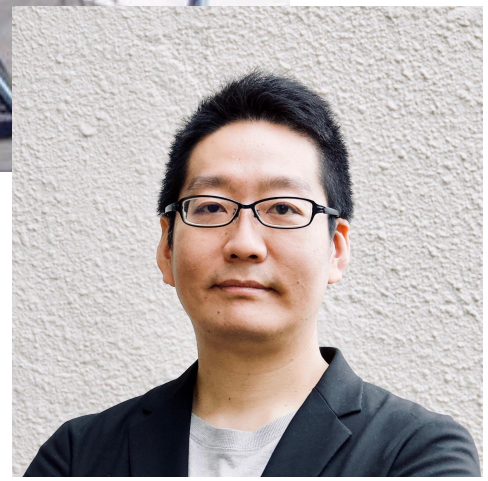
`AGENTS.md` と Agent Skills

OctoNihon Forum#6

Masayuki Matsuki (Songmu)
Senior Solutions Engineer
GitHub Japan



Masayuki Matsuki
Songmu

Senior Solutions Engineer / GitHub Japan

- キャリア
  - 中国での起業
  - 語学学校でオンラインレッスン営業兼シス担
  - 印刷系SIerのSE
  - Web系スタートアップ複数社での技術幹部
  - 昨年12月よりGitHub所属
- 趣味と実益を兼ねたOSS開発者
  - 「OSS作家」を自称
    - 🎙「趣味でOSSをやっているものだ」
  - 200以上のOSSをGitHub上で公開

SD誌 2026年6月号 発売中

- GitHub Copilot特集の第4章を寄稿しました
  - GitHub Copilot CLIについて書いています

最近のOSS貢献

- GitHub Copilotにより捗っている
- OSサンドボックスツールfence: https://fencesandbox.com/ へのpull request
  - https://github.com/jy-tan/fence/pull/110
  - https://github.com/jy-tan/fence/pull/127
- CLIのAI Agentと組み合わせて使うと便利

fix: mach-lookup/mach-register wildcard patterns were completely broken

Merged jy-tan jy-tan:main ← Songmu:fix/mach-wildcard-double-escape +30 -3

Conversation 4 Commits 2 Checks 6 Files changed 2

Songmu commented on Apr 17 • edited Contributor

I'm very happy about the Mach support.

However, wildcard patterns like `"com.apple.*"` or `"md.obsidian.*"` in macos.mach.lookup/register had no effect at all. Only the catch-all `"*"` worked; any glob-style wildcard silently failed to match anything.

The root cause was in `buildMachPermissionRule`: it applied `escapePath()` (which uses `fmt.Sprintf("%q")`) to the regex string. This double-escaped the backslashes produced by `regexp.QuoteMeta`, turning the intended SBPL pattern:

```
(allow mach-lookup (global-name-regex #"^com\.apple\."))
```

into:

```
(allow mach-lookup (global-name-regex #"^com\\.apple\\."))
```

In SBPL regex, `"\\."` means literal-backslash + any-char, so the pattern could never match real service names like com.apple.diagnosticd.

Fix by embedding the regex directly in `#"..."` format without escapePath. Also add test cases for mixed exact+wildcard and star-takes-precedence.

Updating GitHub Copilot Settings

Merged jy-tan jy-tan:main ← Songmu:copilot +2 -4

Conversation 8 Commits 2 Checks 6 Files changed 1

Songmu commented on Apr 11 • edited Contributor

Thank you very much for providing such a truly wonderful tool. I am using it with GitHub Copilot CLI.

Since there were settings related to GitHub Copilot, I assumed you expected it to work. However, the current configuration was not functioning, so I have updated it.

I have updated the URLs for Copilot to match the official documentation. Reference: https://docs.github.com/en/copilot/reference/copilot-allowlist-reference

I changed `api.individual.githubcopilot.com` to `*.individual.githubcopilot.com`. Since it seems to access `telemetry.individual.githubcopilot.com` as well, this should be correct.

The `*.business` and `*.enterprise` URLs are for Copilot Business and Copilot Enterprise.

I added `~/.npm/_npx` to the allowWrite list. This is because copilot-cli runs via npx, and it needs access when performing updates or similar operations.

GitHub Copilot CLI reads settings related to gh. Specifically, it needs access to `~/.config/gh/config.yml` and `~/.config/gh/hosts.yml`. Addtionally, on macOS, gh credentials are stored in the Keychain, and on Linux, they are stored in GNOME Keyring or KDE Wallet. Therefore, this directory does not contain sensitive information.

Copilot CLIを隔離環境で安全に動かす

OSサンドボックス

`$ fence copilot` 等とすることでOSサンドボックス内でCopilot CLIを起動

- 余計なファイルの読み書きやネットワークアクセスをOSレベルで制限
  - Linux, Macのみ対応
- 参考: [fence - AI AgentをOSサンドボックスの中で動かす]

GitHub Copilotもネイティブでサンドボックス機能を開発していると聞いています。

Agenda

- Agentへの指示の最適化
- オープンスタンダード志向
- AIへの指示ファイル形式多すぎ問題
- インストラクションファイル (AGENTS.md)
- Agent Skills
- Agent Plugins
- SkillsやPluginsの管理

Agentへの指示の最適化

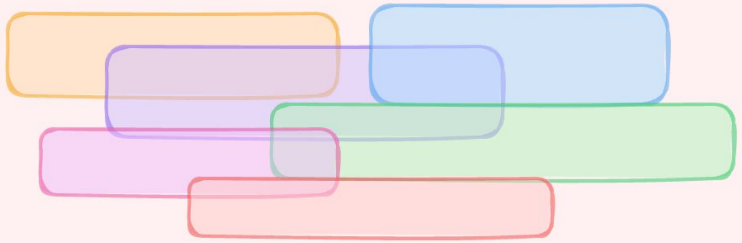
Usage-Based Billing (UBB) への移行

- いわゆるトークン消費量ベースの料金体系への移行
- 色々ご不便をおかけしております…

Token Maxxing から Token Optimization へ

Token Maxxing

大量の雑多なコンテキスト



↓

常に最強モデル

↓

品質にバラつき

- 最強モデルを叩き続ける
- 文脈を限界まで読み込ませる
- 量 = 生産性
- コスト度外視

→

Token Optimization

設計された適切なコンテキスト

タスク定義

必要なコンテキスト

制約・ルール

期待する出力形式

↓

タスクに最適なモデル

↓

高品質なアウトプット

- 必要十分なコンテキストを設計
- 指示の質がアウトプットの質を決める
- タスク × モデルの最適マッチ
- 効率と品質の両立

GitHub Copilot ウェビナー：エージェントの品質とトークンの最適化

6月16日, 5:00 PM — 6:00 PM JST

参加登録

ハーネスエンジニアリングの重要性向上

Agentへインストラクションやスキルを与える

- 本セッションではハーネスの土台を整える前段階の話をします
- Agentへの指示ファイルの形式やAgent Skillsの話
- GitHubの見解というより、私個人の私見も含まれます

オープンスタンダード志向

AI関連情報・アップデート多すぎ問題

- 各社バラバラの仕様を追いかけるのは大変
  - ロックインリスクも
- 極力共通化されていて欲しい
  - 開発者はオープンな規格化を求めるべき

オープンスタンダードを求める合理性

- 一つの知識を複数のツールやサービスで活用・相互運用・乗り換え可能
  - 認知負荷軽減
  - ロックインリスク低減
- オープンな規格の採用に積極的なツールを選択する
  - ベンダー側にサポート圧をかける

ベンダー側の思惑

クローズにしたい・オープンにしたい、様々な思惑が交錯している

- 独自仕様でロックインしてシェアを維持したい
  - 特に成熟業界でのトップベンダー
- オープンなエコシステムを形成してマーケット拡大やシェアの奪還を図りたい
  - 発展途上の業界
  - 複数ベンダーによる連合
    - その中でのイニシアティブの争い

ベンダー独自ファイル配置を避けたい

- 開発におけるツール選択の自由を極力留保したい
  - 不必要なロックインは避けたい
  - OSS開発の場合は特に
- 例えば自分が使わないエディタの設定が置かれていても嬉しくない
  - `.vscode`, `.idea`
  - 自分が使わないコーディングエージェントの設定についても同様

GitHubとAgentHQ

- Copilotは Agentic Coding方面では後発
- オープンスタンダードな規格に乗る動きを見せている
- 3rd party agent機能もある
- 相互運用性の高さを重要視している印象

GitHub Copilotと相互運用性

- `AGENTS.md` サポート
  - `.github/copilot-instructions.md` の代わりに
  - (何なら `CLAUDE.md` や `GEMINI.md` も読む)
- `.agents/skills` サポート
  - `.github/skills` の代わり
  - (何なら `.claude/skills` も読む)
- 参考
  - Adding repository custom instructions for GitHub Copilot - GitHub Docs
  - Adding agent skills for GitHub Copilot - GitHub Docs

カスタムファイル形式多すぎ

Copilotカスタマイズのための各種ファイル

- 参考: [Copilot customization cheat sheet - GitHub Docs]
- やりたいことは分かるが多すぎて混乱することも

内容	ファイルパス	
⭐ リポジトリ全体への指示	`.github/copilot-instructions.md`, `AGENTS.md`	全体に自動適用
特定のファイルパスや言語への指示	`.github/instructions/*.md`	適切に自動適用
定型プロンプトの再利用	`.github/prompts/*.prompt.md`	`/` コマンド呼び出し
カスタムエージェント	`.github/agents/*.md`	自動呼び出し
⭐ Agent Skills	`.agents/skills/*/SKILL.md`	自動呼び出し, `/` コマンド呼び出し
フック機構	`.github/hooks/*.json`	各種イベント後の自動処理呼び出し
MCP設定	`.mcp.json`	

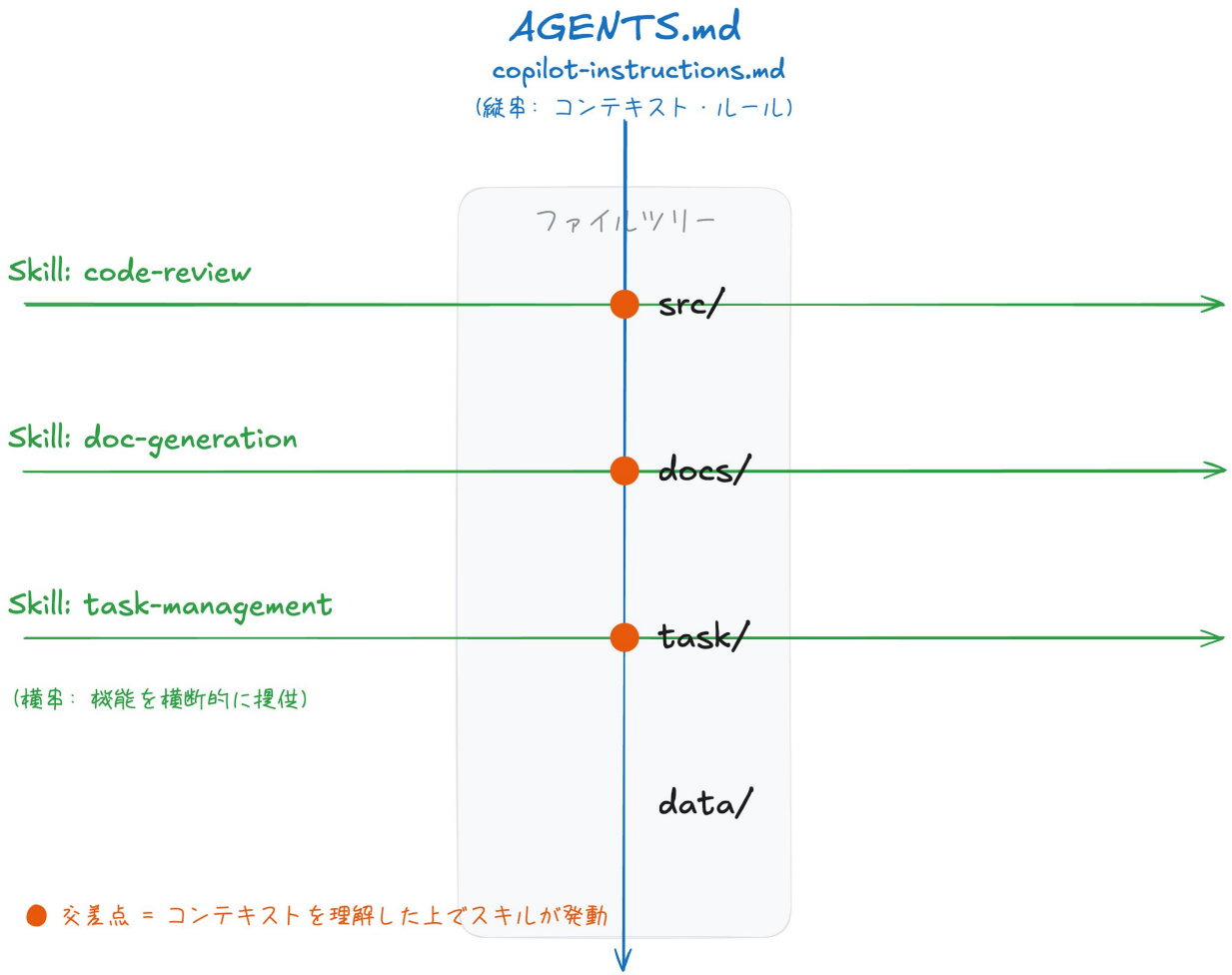
まずは以下の2つだけ意識すればOK

これらに収束していく流れがあり、利用者としてもそれを求めるのが良いのではないか

- インストラクションファイル
  - `AGENTS.md` に配置
- Agent Skills
  - `.agents/skills` に配置

縦串と横串

- インストラクション: ファイルツリーに対するコンテキスト・ルール
- Agent Skills: ファイルの位置を問わず、特定の機能を横断的に提供



あっても最小限に

最初はインストラクションもskillsも無くても良い

- ハーネス整備に時間をかけてAIを使わないのは本末転倒
  - 自分で使ってみて経験知・身体知を獲得する
- 無駄なインストラクションやSkillsを作りすぎるのも無駄
- 最近のLLMはかなり賢い
  - 彼らがすでに知っていることは与えなくて良い
  - コードベースから拾える情報は与えなくて良い
- 余計なトークン消費やハルシネーションリスクを避ける

必要になってからインストラクションやSkillsを追加

- 置き場所は理解しておく
- なんなら、インストラクションやskillsもAIに作ってもらう
  - 「`AGENTS.md` に追加して」
  - 「今の作業をAgent Skills化して」
- AIに自律的に改善してもらうのが理想
  - チームマネジメントと同じ
- 作り過ぎに注意

インストラクションファイル (AGENTS.md)

AGENTS.md

- https://agents.md
- AI Agentへのインストラクションのためのオープンフォーマット
  - OpenAI の Codex CLIで使われていたものをオープンスタンダード化
  - Copilotも追随してサポート

AGENTS.md

A simple, open format for guiding coding agents,
used by over 60k open-source projects.

Think of AGENTS.md as a **README for agents**: a dedicated, predictable place to provide the context and instructions to help AI coding agents work on your project.

Explore Examples

View on GitHub

```
# AGENTS.md

## Setup commands
- Install deps: `pnpm install`
- Start dev server: `pnpm dev`
- Run tests: `pnpm test`

## Code style
- TypeScript strict mode
- Single quotes, no semicolons
- Use functional patterns where possible
```

AGENTS.md を積極的に使う

- 記述内容は `.github/copilot-instructions.md` に書くものと基本的に同じで良い
- 書く量も必要最小限に
- サブディレクトリにも配置できるのはメリット
  - `docs/AGENTS.md`, `src/AGENTS.md` 等

`copilot-instructions.md` の使い所

- Copilot固有のインストラクションを置く場所としては必要
  - 例えば Copilot Code Review は `AGENTS.md` をまだ読まない

例:
コードレビュー時には必ず日本語でレビューを行ってください。すべてのコメント、提案、説明は日本語で記述してください。
Copilot Chat上でのやり取りも日本語で行うようにしてください。

`**.instructions.md` のパス指定

- ex. `applyTo: "docs/**"`, `applyTo: "**/*.ts,**/*.tsx"`
- `AGENTS.md` や Skills に振り分けられるものはそちらに寄せる
  - `applyTo: "docs/**"` -> `docs/AGENTS.md`
  - `applyTo: "**/*.ts,**/*.tsx"` -> TypeScript用のAgent Skills

(備考) 各種インストラクションの適用範囲

各エディタ拡張、CLI, Code Review, Cloud Agent等でサポートされるインストラクションが異なり複雑

- 以下を参照
  - [Support for different types of custom instructions - GitHub Docs]
- GitHub Copilot app (preview) は Copilot CLIと同等と想像
- この辺りも統一されていく未来を期待

GitHub.com

Copilot feature	Types of custom instructions supported
Copilot Chat	**Personal** instructions. **Repository-wide** instructions (using the `.github/copilot-instructions.md` file). **Organization** instructions.
Copilot cloud agent	**Repository-wide** instructions (using the `.github/copilot-instructions.md` file). **Path-specific** instructions (using `.github/instructions/**/*.instructions.md` files). **Agent** instructions (using `AGENTS.md`, `CLAUDE.md` or `GEMINI.md` files). **Organization** instructions.
Copilot code review	**Repository-wide** instructions (using the `.github/copilot-instructions.md` file). **Path-specific** instructions (using `.github/instructions/**/*.instructions.md` files). **Organization** instructions.

Visual Studio Code

Copilot feature	Types of custom instructions supported
Copilot Chat	**Repository-wide** instructions (using the `.github/copilot-instructions.md` file). **Path-specific** instructions (using `.github/instructions/**/*.instructions.md` files). **Agent** instructions (using an `AGENTS.md` file).
Copilot cloud agent	**Repository-wide** instructions (using the `.github/copilot-instructions.md` file). **Path-specific** instructions (using `.github/instructions/**/*.instructions.md` files). **Agent** instructions (using `AGENTS.md`, `CLAUDE.md` or `GEMINI.md` files).
Copilot code review	**Repository-wide** instructions (using the `.github/copilot-instructions.md` file).

Claude CodeとAGENTS.md

AGENTS.mdは主要ベンダーがネイティブサポートしているがClaude Codeはサポートを避けている

- コミュニティからの要望も強い
  - Feature Request: Support AGENTS.md. #6235 - anthropics/claude-code

https://code.claude.com/docs/ja/memory#agents-md

Claude Code は `CLAUDE.md` を読みます。 `AGENTS.md` ではありません。リポジトリが既に他のコーディングエージェント用に `AGENTS.md` を使用している場合、 `CLAUDE.md` を作成してそれをインポートし、両方のツールが重複なしに同じ指示を読むようにします。Claude 固有の指示をインポートの下に追加することもできます。Claude はインポートされたファイルをセッション開始時に読み込み、その後残りを追加します。

CLAUDE.md

```
@AGENTS.md

## Claude Code

`src/billing/` の下の変更には Plan Mode を使用します。
```

シンボリックリンクも機能します。Claude 固有のコンテンツを追加する必要がない場合は、次のようにします。

```
ln -s AGENTS.md CLAUDE.md
```

Windows では、シンボリックリンクを作成するには管理者権限または開発者モードが必要なため、代わりに `@AGENTS.md` インポートを使用します。

Agent Skills

Agent Skillsとは

- Agentに特定タスクの専門知識・手順を教える機能
  - `.agents/skills/<スキル名>/SKILL.md` に配置
- プロンプトの内容に応じてCopilotが自動的にマッチしたスキルを読み込む
  - slashコマンドでの実行も可
- カスタムAgentは専門家的なペルソナ、Skillsはそれに道具を渡すイメージ

参考:
https://github.github.io/awesome-copilot/learning-hub/what-are-agents-skills-instructions/

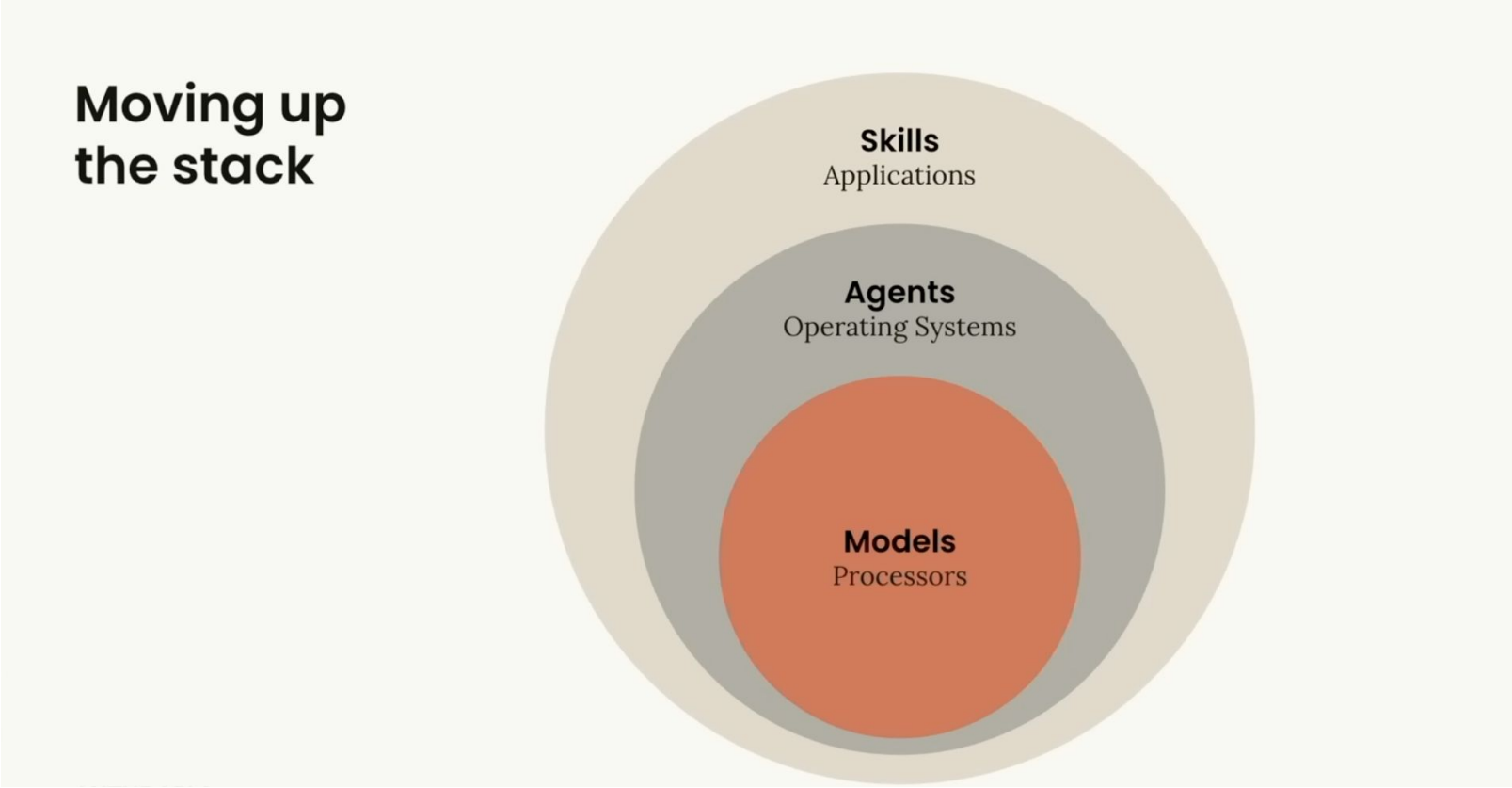
Agent Skillsのエコシステム

- オープンスタンダードとして進化中: https://agentskills.io
  - 元々はAnthropic社が提唱、GitHub Copilotも採用している
- 参考
  - https://claude.com/blog/equipping-agents-for-the-real-world-with-agent-skills
  - https://code.claude.com/docs/ja/skills
  - skill-creatorから学ぶSkill設計と、Orchestration Skillの作り方
  - What are Agents, Skills, and Instructions

Models, Agents, Skillsの関係概念図

Don't Build Agents, Build Skills Instead – Barry Zhang & Mahesh Murag, Anthropic より

ただ、これは出てきた当初の図で、Skills自体が複数のagentを動かすようなオーケストレーターとして動くケースも出てきたので入れ子的になってきてはいる。



SKILL.mdの書き方

- YAMLフロントマター（`name`, `description`）＋ Markdown本文の構成
- `description` がマッチングのキーになるため、いつ使うかを明記
- 参考: [GitHub Docs: Creating agent skills]
- 参考: [VS Code Docs: Agent Skills]

```
---
name: create-readme
description: 'Create a README.md file for the project'
---

## Role

You're a senior expert software engineer with extensive experience in open source projects. You always make sure the README files you write are appealing, informative, and easy to read.

## Task
...
<!-- https://github.com/github/awesome-copilot/blob/main/skills/create-readme/SKILL.md より引用 -->
```

Skillsへの統合の流れ

各社のカスタマイズファイルがSkillsに統合され、1本化されつつある

- Claude CodeのSlash CommandはSkillsへ
  - Merging Slash Commands into Skills in Claude Code
- CopilotもPromptsからSkillsへ
  - Awesome Copilotリポジトリでは PromptsをSkillsに統合する移行が実施済
  - Copilot CLIはPrompts未サポート
  - What are Agents, Skills, and Instructions | awesome-copilot
    - "Why skills over prompts"
    - Skillsはリソース同梱・Agent自動発見・クロスプラットフォーム対応で上位互換

Slash CommandもPromptsも使えなくなるわけではなく互換は維持される

PromptsとSkillsの現状の非互換

`model`, `tools` 指定や変数を活用している場合、Skillsへの移行を急がなくても良い。

	Prompts	Skills
Agent自動トリガー	❌ 手動のみ	✅ descriptionでマッチ
リソース同梱	❌ 単一ファイル	✅ scripts/, references/
`model` / `tools` 指定	✅	❌（仕様に未実装）
`${file}` 等の変数	✅	❌（仕様に未実装）
クロスプラットフォーム	❌ Copilot専用	✅ オープン仕様

Agent Skills仕様の議論状況

Promptsにある機能	Skills仕様での状況	関連Issue
`tools` 指定	`allowed-tools` として存在（experimental）	#144 型の明確化
`model` 指定	未実装	Discussion #150
`${file}` 等の変数	未実装	#124 動的コンテキスト注入

自動トリガー周りの制御

- `/` コマンドだけで明示的に呼び出したい場合
  - `disable-model-invocation: true` でAgent自動トリガーを無効化可能
  - デフォルトは `false`
- 逆に、ユーザーが明示的に呼び出さないことを前提にしたい場合
  - `user-invocable: false` でユーザーからの呼び出しを無効化可能
  - デフォルトは `true`

PromptsもSlash CommandもSkillsに統合されつつある

カスタムAgentsはどうなるか

Copilotの `.github/agents/CUSTOM-AGENT-NAME.md` や Claude Codeの `.claude/agents/CUSTOM-AGENT-NAME.md` 等、特定のペルソナや専門家の役割を与えるためのファイル

- これもSkillsに寄っていくのではないか (私見)
- ペルソナを指定するプロンプトは余り意味が無いと認識されつつある
  - ex.「あなたは〇〇の専門家です」
  - それっぽく振る舞うだけでアウトプット品質はあまり変わらない
  - 関西弁話者などを模してもらって楽しむ分には面白いが
- 役割やペルソナを与えるのではなく、ゴールやタスクを与える方が有用
  - であれば結局Skillsにすれば良いのでは
  - ちなみにCodexにはカスタムエージェントの機能は無い

Skillsを作るか・有物を使うか

- まずは自分や自分たちで作ってみるのも良いのではないか
  - 「今の作業をAgent Skill化して」
- オープンに公開されているものも多い
  - 使う場合には安全性やライセンスの検証が必須

絶対ダメ →「便利そうなものをバンバンインストールする」

`gh skill` でのSkillsインストール

- GitHubリポジトリの `skills/` ディレクトリに配置されたAgent SkillsをGitHub CLIからインストール可能
  - [Manage agent skills with GitHub CLI - GitHub Changelog]
  - Copilot用途に限らず利用可能
- タグやハッシュでのpin留めやアップデートにも対応
  - リポジトリのコミットやタグによるバージョニング

```
# Browse skills in a repository and install them interactively
gh skill install github/awesome-copilot

# Or install a specific skill directly
gh skill install github/awesome-copilot documentation-writer

# Install a specific version using @tag
gh skill install github/awesome-copilot documentation-writer@v1.2.0

# Install at a specific commit SHA
gh skill install github/awesome-copilot documentation-writer@abc123def

# Discover skills
gh skill search mcp-apps
```

課題

- Skillsは強力である反面リスクもはらむ
  - 安全な配布
  - ガバナンス
- 管理方法などもまだ未成熟
  - リリース・パッケージング
  - バージョニング・バージョンロック
  - 階層・名前空間
  - 依存管理・コンフリクト解消
- 危険だからこそ共通化して業界で解決
  - リスクの局所化
  - コミュニティリポジトリ上で議論はされている

Agent Plugins

Agent Plugins

- 各種カスタムファイルの再利用性を高めるための仕組
- Agent Skills, Hooks, MCP設定などをパッケージングして配布するための機能
  - プラグイン自体の定義方法 (`plugin.json`)
  - 配布のためのマーケットプレイス (`marketplace.json`)
- GitHub Copilot CLIで先行実装されていたものがVS Codeにも展開
  - About plugins for GitHub Copilot CLI - GitHub Docs
  - Agent plugins in VS Code (Preview)

ディレクトリ構造と `plugin.json` の中身

Pluginの構造

- ディレクトリをPluginとして管理する
- `plugin.json` でプラグインのメタデータや構成を定義

```
my-plugin/
├── plugin.json              # プラグインメタデータ（必須: name）
├── skills/
│   └── test-runner/
│       ├── SKILL.md          # スキル定義
│       └── run-tests.sh      # 補助スクリプト
├── agents/
│   └── reviewer.agent.md     # カスタムエージェント
├── hooks.json               # ライフサイクルフック設定
├── scripts/
│   └── validate.sh           # フックから呼ばれるスクリプト
└── .mcp.json                # MCP サーバー定義
```

```
// plugin.json
{
  "name": "my-dev-tools",
  "version": "1.2.0",
  "skills": "skills/",
  "agents": "agents/",
  "hooks": "hooks.json",
  "mcpServers": ".mcp.json"
}
```

Claude CodeのPlugin機構とほぼ同様

https://code.claude.com/docs/ja/plugin-marketplaces

- 完全互換ではない

PLUGIN 配布

プラグインマーケットプレイスの作成と配布

ページをコピー

Claude Code 拡張機能を配布するためのプラグインマーケットプレイスを構築およびホストします。

プラグインマーケットプレイスは、他のユーザーにプラグインを配布できるカタログです。マーケットプレイスは、一元化された検出、バージョン追跡、自動更新、および複数のソースタイプ（Git リポジトリ、ローカルパス、その他）のサポートを提供します。このガイドでは、チームやコミュニティとプラグインを共有するための独自のマーケットプレイスを作成する方法を説明します。

既存のマーケットプレイスからプラグインをインストールしたいですか？**既成プラグインの検出とインストール**を参照してください。

Claude CodeのプラグインはCopilotで利用可能

[About plugins for GitHub Copilot CLI - GitHub Docs]

Where can I get plugins?

You can install plugins from:

- A marketplace
- A repository
- A local path

A marketplace is a location where developers can publish, discover, install, and manage plugins. It's a bit like an app store—but for plugins.

Examples of marketplaces include:

- copilot-plugins (added by default)
- awesome-copilot (added by default)
- claude-code-plugins
- claudeforge-marketplace

For more about adding marketplaces and installing plugins from them, see Finding and installing plugins for GitHub Copilot CLI.

Pluginのインストール

- スラッシュコマンドでインストール可能
  - VSCode, CLI
- Plugin Marketplacesからのインストール
- 標準で2つのMarketplaceが登録
  - `github/copilot-plugins`
  - `github/awesome-copilot`

```
/plugin install marketplace@plugin-name
```

独自マーケットプレイスの作成

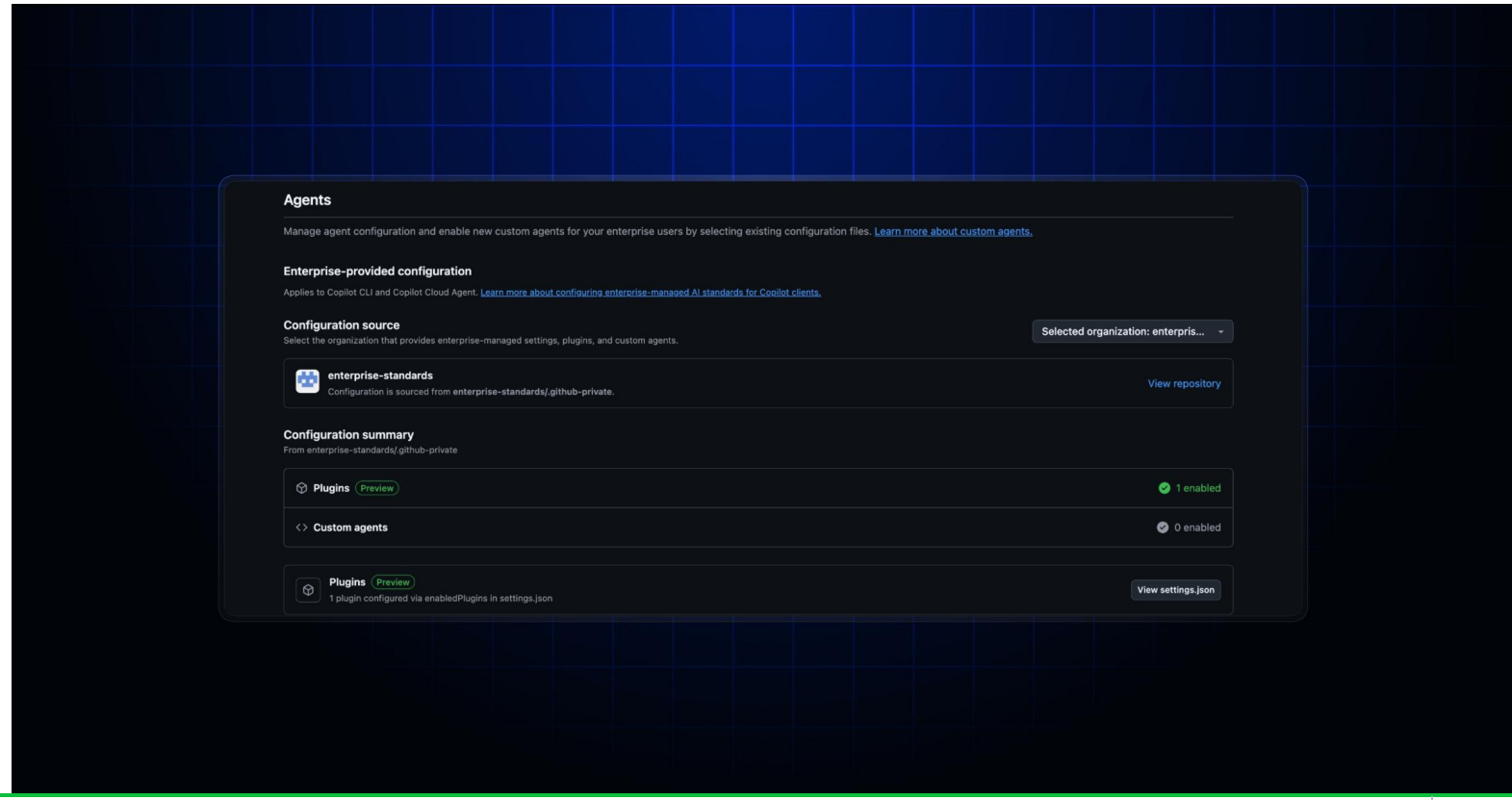
- Gitリポジトリに `marketplace.json` を配置するだけで作成可能
  - プライベートリポジトリ可

```
// .github/plugin/marketplace.json
{
  "name": "my-marketplace",
  "owner": {
    "name": "Your Organization",
    "email": "plugins@example.com"
  },
  "metadata": {
    "description": "Curated plugins for our team",
    "version": "1.0.0"
  },
  "plugins": [
    {
      "name": "frontend-design",
      "description": "Create a professional-looking GUI ...",
      "version": "2.1.0",
      "source": "./plugins/frontend-design"
    },
    {
      "name": "security-checks",
      "description": "Check for potential security vulnerabilities ...",
      "version": "1.3.0",
      "source": "./plugins/security-checks"
    }
  ]
}
```

SkillsやPluginsの管理

エンタープライズ向けプラグイン管理 (preview)

- エンタープライズレベルで利用可能なプラグインを管理する機能
- 現状Preview機能でCLIのみサポート
  - About enterprise-managed plugin standards for Copilot CLI - GitHub Enterprise Cloud Docs
  - Enterprise-managed plugins in GitHub Copilot CLI are now in public preview - GitHub Changelog



Enterprise-managed plugins利用方法

- 管理用の `.github-private` リポジトリを作成
- `.github/copilot/settings.json` に以下のようなファイルを配置
- まだ機能的には不十分
  - [Configuring enterprise plugin standards for Copilot CLI - GitHub Enterprise Cloud Docs]

```
{
  "extraKnownMarketplaces": {
    "MARKETPLACE-NAME": {
      "source": {
        "source": "github",
        "repo": "OWNER/REPO"
      }
    }
  },
  "enabledPlugins": {
    "PLUGIN-NAME@MARKETPLACE-NAME": true
  }
}
```

プロジェクト内でのAgent SkillsやPluginsの依存管理

APM - Agent Package Manager

https://github.com/microsoft/apm

- `apm.yml` と言うマニフェストファイルで宣言
- `apm install` コマンドでの一元インストール
- `apm.lock.yaml` でバージョンロック

Microsoft orgで公開されているがGitHubとしてまだ強く推している訳では無い

apm.ymlの例

公式リポジトリより引用

```yaml
# apm.yml — ships with your project
name: your-project
version: 1.0.0
dependencies:
  apm:
    # Skills from any repository
    - anthropics/skills/skills/frontend-design
    # Plugins
    - github/awesome-copilot/plugins/context-engineering
    # Specific agent primitives from any repository
    - github/awesome-copilot/agents/api-architect.agent.md
    # A full APM package with instructions, skills, prompts, hooks...
    - microsoft/apm-sample-package#v1.0.0
  mcp:
    # MCP servers -- installed into every detected client
    - name: io.github.github/github-mcp-server
      transport: http   # MCP transport name, not URL scheme -- connects over HTTPS
```

APM所感

- こういったものは絶対に必要
  - やろうとしていることの網羅性も高い
  - 最近のサプライチェーンアタック怖すぎるので…
- 各AI Agent間の差異を吸収するような機能も備えるが
  - ターゲットによってインストール先を変えるなど
  - 「がんばりすぎ」感もある
    - 仕様を統一していくか・頑張って吸収するのか
- コミュニティ・エコシステム発祥というより実装先行
  - コミュニティにどれくらい受け入れられるかはまだ未知数
  - 最近話を聞くようにはなってきた
- 動作パフォーマンス的にはあまり良くないとも聞く
  - 各Gitリポジトリから取得する設計なので遅くなるのは仕方なさそう
- 動向に注目
  - 定番ツールが早く何か定着してほしい

まとめ

縦串の`AGENTS.md`と横串の`Agent Skills`

- オープンスタンダードな`AGENTS.md`と`.agents/skills`への配置が個人的推奨
  - GitHub Copilotも対応しています
- Agent Skillsやその他アセットのサプライチェーンマネジメントは重要な課題
  - APMなども出てきている
  - 複数リポジトリでの設定共有などにも課題感あり

Thank you